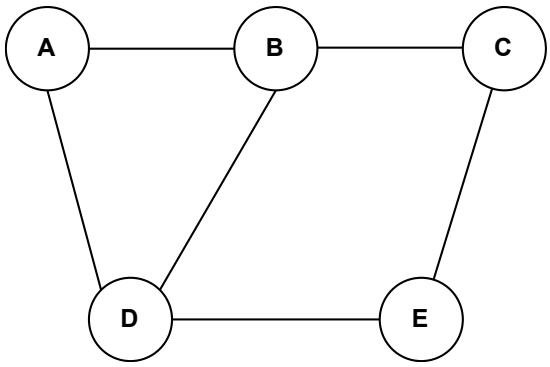
# Introduction to Graph

A graph is a data structure that consists of a set of vertices (nodes) and a bunch of edges connecting these vertices. Graphs are widely used to model real-world scenarios where relationships and connections between entities must be represented. A Graph G (V, E) with five vertices (A, B, C, D, E) and six edges ((A, B), (B, C), (C, E), (E, D), (D, B), (D, A)) are shown in the following figure.




Graphs can become more complex as they grow and have many interconnected nodes and edges. Graphs are a fundamental and versatile data structure used in various computer science applications to model and solve complex problems involving relationships and connections.

## Application of Graphs

Graphs are versatile data structures that find applications in various fields. Here are five graph applications with facts and figures:

**1. Social Network Analysis**: Graphs are widely used for analyzing social networks to understand connections between individuals or entities. One of the most popular social networks is Facebook.

**2. Transportation Network Optimization**: Graphs are crucial in optimizing transportation networks like road or flight routes. For example, the road network of the United States spans over 4 million miles. Using graph algorithms like Dijkstra's algorithm, authorities can efficiently find the shortest path between two locations, minimizing travel time and fuel consumption.

**3. Internet and Webpage Ranking**: Search engines like Google use graphs to rank webpages and determine their relevance to a search query. Google's PageRank algorithm, which Larry Page and Sergey Brin introduced, utilizes a graph representation of the web.

**4. Recommendation Systems**: Graphs are employed in recommendation systems to provide personalized suggestions to users based on their preferences and behaviors. A popular streaming platform, Netflix uses a graph-based collaborative filtering approach to recommend movies and TV shows to its subscribers.

**5. Bioinformatics and Protein Interactions**: In bioinformatics, graphs model complex biological interactions, such as protein-protein interactions, gene regulatory networks, and metabolic pathways. The Human Protein Reference Database (HPRD), which stores protein-protein interaction data, contains information about more than 39,000 interactions among over 9,000 proteins.

These examples demonstrate the wide-ranging applications of graphs in different domains, each significantly impacting the respective industries.

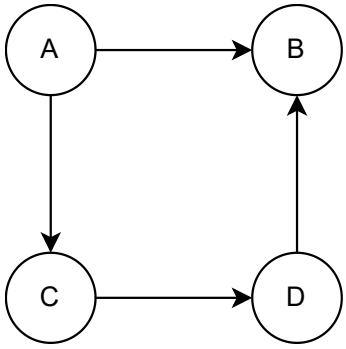
## Common Graph Terminologies

Understanding baseline graph terminologies is essential for understanding complex graph concepts. Like the other concepts, graph theory has some baseline terminologies. These terminologies are the most commonly used jargon in graphs. Understanding these jargons empower you better understand and communicate the graph concept.

Here are some of the most commonly used graph terminologies:

**1. Digraph**:

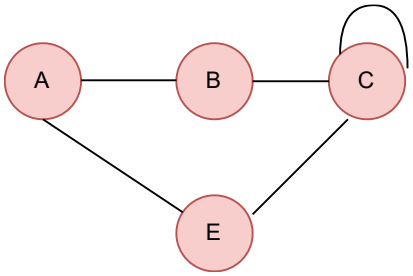
A digraph, short for directed graph, is a type of graph in which edges have a direction or are represented by arrows. Each edge connects two vertices (nodes), but the direction of the edge indicates a one-way relationship between the nodes. In other words, if there is an edge from vertex A to vertex B, you can only travel from A to B along that edge.



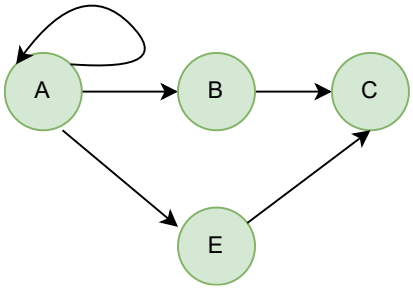

**3. Loop in Graph**:

A "loop" in a graph refers to an edge that connects a vertex to itself. In other words, it is an edge that starts and ends at the same vertex. Loops can exist in both directed and undirected graphs.

In an undirected graph, a loop is simply an edge that connects a vertex to itself. It forms a cycle of length 1. For example, if you have a graph with a single vertex v, and there is an edge from v to itself, then it forms a loop.

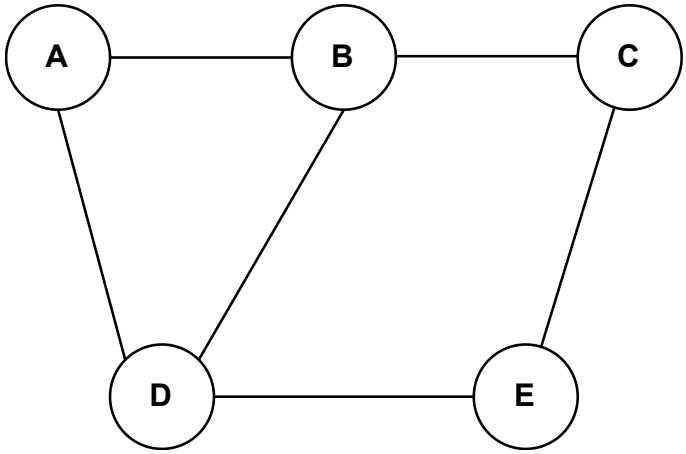



In a directed graph, a loop is an arc (directed edge) that starts and ends at the same vertex. It forms a directed cycle of length 1. For example, if you have a graph with a single vertex v, and there is a directed edge from v to itself, then it forms a loop.




**4. Node in Graph**:

A "node" (also known as a "vertex") is a fundamental building block of a graph. A graph is a mathematical representation of a set of vertices (nodes) and the connections (edges) between those objects. The links between nodes can be either directed or undirected, depending on whether the relationship has a specific direction. In the following figure, A, B, C, D, and E are the nodes in the graph, also known as vertices.




**5. Adjacent Nodes**:

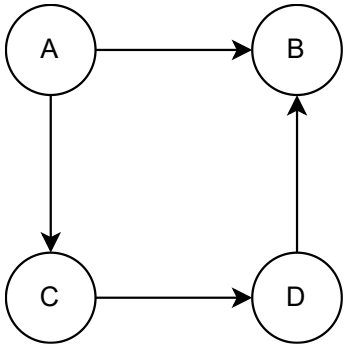
Two nodes are said to be adjacent if an edge is connected directly. The set of nodes adjacent to a particular node is known as its neighborhood. As in the above figure, the nodes A and B are adjacent.

**6. Degree of a Node**:

The degree of a node is the number of edges connected to it. In an undirected graph, it represents the number of neighbors a node has. As in the above figure, the degree of Node A is two because it has two neighbors.

In a directed graph, there are two degrees:

- The in-degree (number of incoming edges)
- The out-degree (number of outgoing edges).



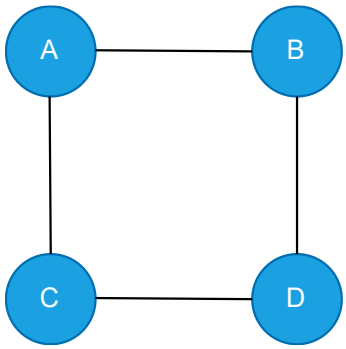

In the above graph, the degree of node A is two because two edges are outgoing, and the degree of node C is 1.

**7. Path**:

A sequence of vertices in which each consecutive pair of vertices is connected by an edge. For example in the above figure, there is a path from vertex A to B.

**8. Cycle**:

A path in which the first and last vertices are the same, forming a closed loop. The following figure shows a route from A to B, B to D, D to C, and C to A. So here, the cycle completes because we start from vertex A and end again at A vertex.

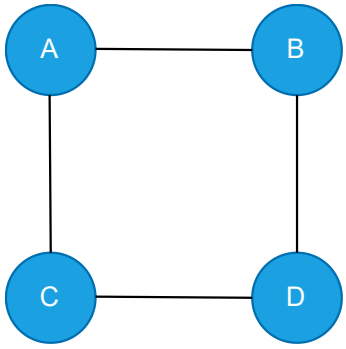



## Graph Types

There are several types of graphs, each with its specific characteristics. Here are some common types of graphs, along with examples:

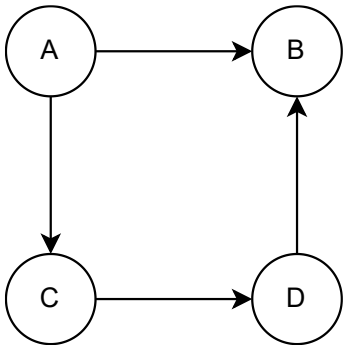
**1. Undirected Graph**: In an undirected graph, edges have no direction, representing a bidirectional connection between two vertices. If an edge exists between vertex A and vertex B, you can traverse from A to B and vice versa.

Example: Friends Network



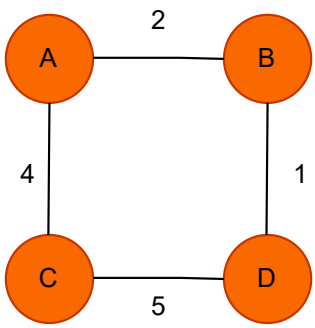

**2. Directed Graph**: In a directed graph, edges have a direction, indicating a one-way connection between vertices. If there is an edge from vertex A to vertex B, you can only traverse from A to B, not vice versa.

Example: Webpage Links



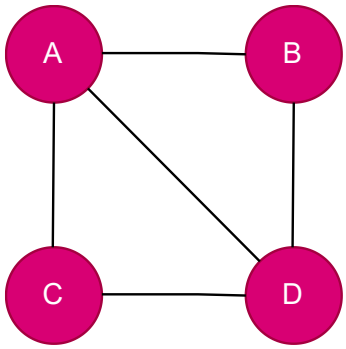

**3. Weighted Graph**: Each edge is associated with a numerical value called a weight in a weighted graph. The weight can represent distances, costs, or any other relevant metric between the connected vertices.

Example: Transportation Network

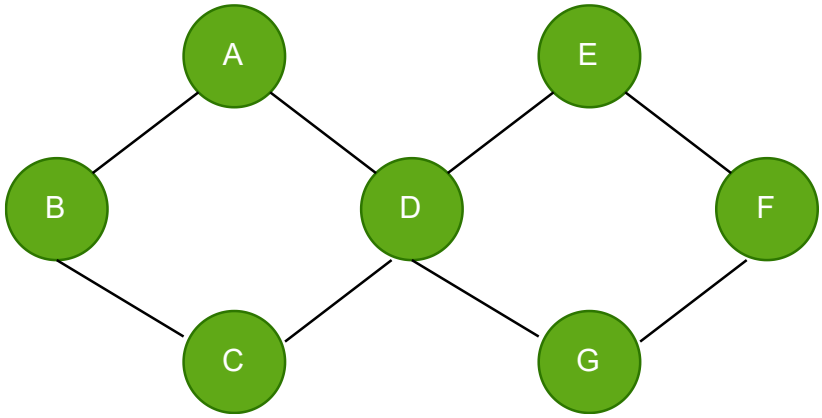


**4. Unweighted Graph**: All edges have the same default weight of 1 in an unweighted graph. There are no additional numerical values associated with the edges. In an unweighted graph, the absence of edge weights implies that all edges are considered to have equal importance or distance between the connected nodes.

Example: Family Tree (Connection exists or doesn't)




**5. Cyclic Graph**: A cyclic graph is a graph that contains at least one cycle, which is a closed path (sequence of vertices) that starts and ends at the same vertex.

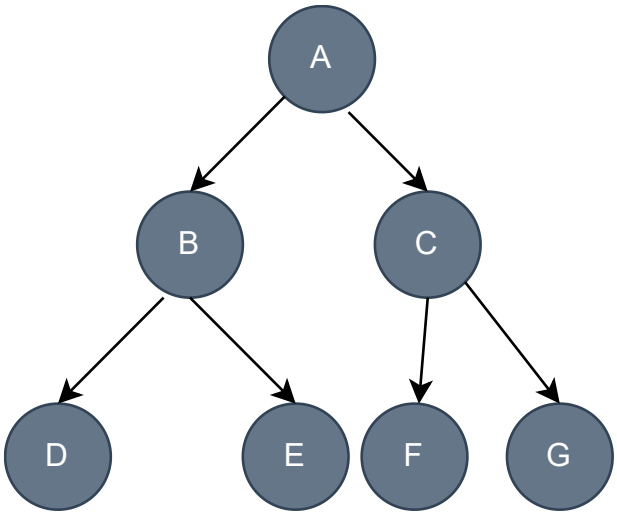



**6. Acyclic Graph**: An acyclic graph is a directed graph that has no cycles. A cycle occurs when the following edges from a node lead back to the same node. Some key properties of acyclic graphs:

- They have at least one node with no incoming edges (called a source node).
- They have at least one node with no outgoing edges (called a sink node).

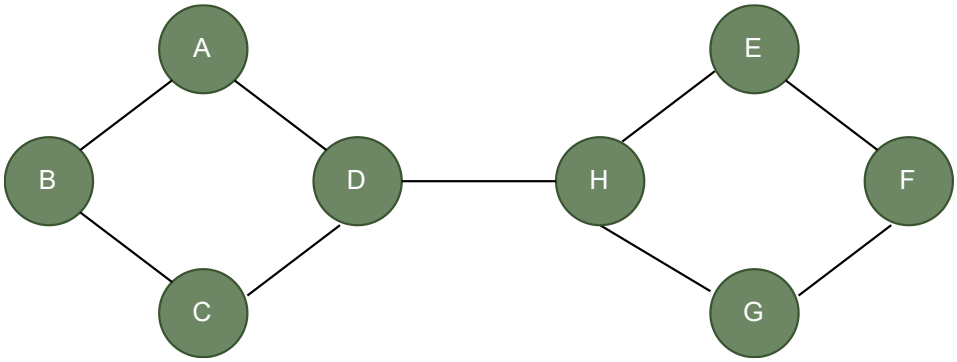
  In this graph, node A is the root node. It has no incoming edges. Nodes D, E, F, G are leaf nodes - they have no outgoing edges. There are no cycles in this graph.

A valid topological ordering of the nodes could be: A, B, C, D, E, F, G. So this graph structure forms an acyclic-directed graph. Trees and DAGs (Directed Acyclic Graphs) are common examples of acyclic graph structures



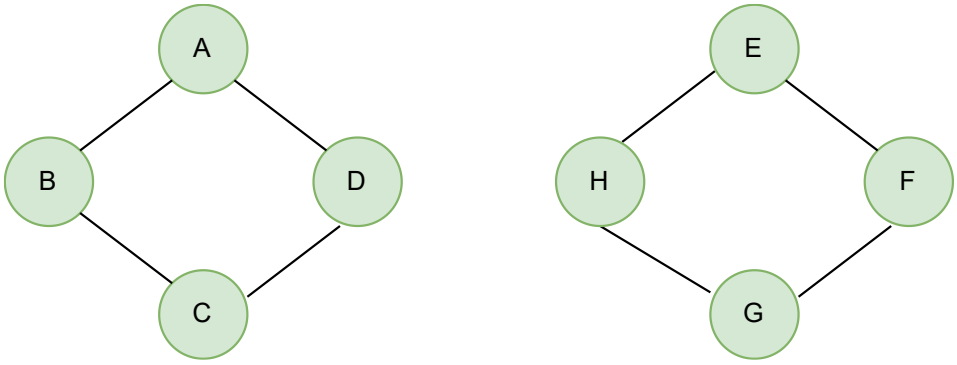

Acyclic Graph

**7. Connected Graph**: A connected graph is one in which there is a path between every pair of vertices. In other words, every vertex is reachable from any other vertex in the graph.



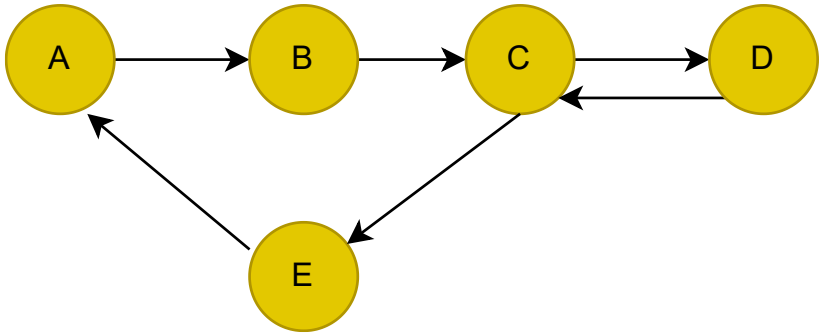

Connected Graph

**8. Disconnected Graph**: A disconnected graph has two or more connected components (subgraphs) with no direct connection between these components. The figure below is one separate graph. The first component contains A, B, C, and D vertices, and the other part contains E, F, G, and H, with at least two vertices not connected by a path.




Disconnected Graph

**9. Strongly Connected Graphs**: A strongly connected graph is a type of directed graph in which there is a directed path from every vertex to every other vertex. In other words, for any two vertices, A and B, in a strongly connected graph, there is a directed path from A to B and B to A.




Strongly Connected Graphs

These are some of the common types of graphs in data structures. Each type has its significance and use cases depending on the problem you're trying to solve. Understanding different types of graphs is crucial for effectively applying graph algorithms and solving graph-related problems.

← Previous

Solution: Problem Challenge 2: Path With Maximum Sum

Next →

Graph Representations

Mark as Completed